

Resolución del Práctico 6

Testing de Software 2026

Tomás Rodeghiero

Índice

1. Ejercicio 1: mutación sobre <code>Palindrome.capicua</code>	2
1.1. El método	2
1.2. Ejecución de PIT	2
1.3. Suite	2
1.4. Respuestas	3
1.5. Cierre	4
2. Ejercicio 2: mutación sobre <code>TriTyp.triang</code>	4
2.1. Ejecución de PIT	4
2.2. Suite	4
2.3. Respuestas	5
2.4. Cierre	5
3. Ejercicio 3: fuzzing y <code>bc</code>	6
3.1. Mutator	6
3.2. MutationFuzzer	6
3.3. RandomFuzzer	7
3.4. LinuxCommandTest	7
3.5. Verificación	8

1. Ejercicio 1: mutación sobre `Palindrome.capicua`

Testing de mutación con PIT sobre `Palindrome.capicua`. La consigna pide generar mutantes, escribir una suite JUnit que los mate, y responder cuatro preguntas (cantidad de mutantes, tests, mutation score y mutantes equivalentes).

1.1. El método

`Palindrome.capicua(char[])` recibe un arreglo de `char` y determina si es capicúa recorriéndolo desde los extremos hacia el centro mientras los caracteres coincidan:

```
1 public static boolean capicua(char[] list) {
2     int index = 0;
3     int l = list.length;
4     boolean res = true;
5     while (index < (l - 1) && res) {
6         if (list[index] != list[(l - index) - 1]) {
7             res = false;
8         }
9         index++;
10    }
11    return res;
12 }
```

1.2. Ejecución de PIT

Desde `tp6/assignment-6-rodeghiero`:

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true test
    org.pitest:pitest-maven:mutationCoverage
```

`-Djacoco.skip=true` evita conflictos con JaCoCo (PIT instrumenta el bytecode por su cuenta).

1.3. Suite

5 tests pensados para cubrir caminos típicos y bordes:

1. `testCapicua()` — capicúa típica ("neuquen").
2. `testNoCapicuaPar()` — caso mínimo no capicúa con dos caracteres distintos ({'a', 'b'}).

3. `testNoCapicuaConExtremosIguales()` — `{'a','b','c','a'}`. Es el **test clave**: obliga a que la verificación no se quede solo en comparar el primer y último carácter.
4. `testArregloVacioEsCapicua()` — arreglo vacío (capicúa por definición).
5. `testUnElementoEsCapicua()` — arreglo de un elemento (capicúa trivialmente).

1.4. Respuestas

(a) ¿Cuántos mutantes generó la herramienta? PIT generó **9 mutantes** sobre capicua.

(b) ¿Cuántos tests definí? **5 tests**, suficientes para matar todos los mutantes no equivalentes.

(c) **Mutation score. 89 % (8/9)**. Progreso de la suite:

- Línea base inicial (suite del template): 56 % (5/9).
- Suite final: 89 % (8/9).

(d) **Mutantes equivalentes**. Hay **1 mutante equivalente**: una mutación de frontera en la guarda del `while`:

- Original: `index < (1 - 1)`
- Mutado: `index <= (1 - 1)`

El cambio no altera el resultado observable en ninguna entrada:

1. Si `1 == 0`, ninguna versión entra al `while`, ambas devuelven `true`.
2. Si `1 == 1`, la versión mutada agrega una iteración extra que compara `list[0]` consigo mismo. Esa comparación siempre da verdadera y `res` no cambia.
3. Si `1 >= 2`, la iteración adicional con `index = 1 - 1` vuelve a comparar un par de extremos que ya había sido evaluado antes (en posición espejo). Tampoco cambia `res`.

No existe entrada que diferencie el código original del mutado, así que no se puede matar sin cambiar la semántica.

1.5. Cierre

Métrica	Valor
Mutantes generados	9
Mutantes muertos	8
Mutantes equivalentes	1
Score bruto	89 %
Score sobre no equivalentes	100 %

2. Ejercicio 2: mutación sobre `TriTyp.triang`

Testing de mutación con PIT sobre `TriTyp.triang`, que clasifica un triángulo como equilátero, isósceles, escaleno o no-triángulo a partir de tres lados.

2.1. Ejecución de PIT

El `pom.xml` apunta a `Palindrome` por defecto, así que para correr Pitest sobre `TriTyp` hay que sobrescribir `targetClasses` y `targetTests`:

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true \  
    -DtargetClasses=assignment6_exercises.TriTyp \  
    -DtargetTests=assignment6_exercises.TriTypTest \  
    test org.pitest:pitest-maven:mutationCoverage
```

2.2. Suite

15 tests que activan todas las decisiones relevantes de `triang` y, por extensión, todos los mutantes generados por PIT. Los casos cubren:

1. Triángulo equilátero válido.
2. Triángulo escaleno válido.
3. Escaleno que **no** forma triángulo por igualdad estricta de la suma de dos lados con el tercero (tres permutaciones).
4. Escaleno que no forma triángulo por desigualdad estricta (la suma de dos lados es menor que el tercero).
5. Isósceles válido en sus tres variantes ($S1 == S2$, $S1 == S3$, $S2 == S3$).
6. Isósceles que no forma triángulo en cada una de esas tres variantes.
7. Casos con al menos un lado no positivo, evaluando cada una de las tres posiciones.

2.3. Respuestas

(a) PIT generó **39** **mutantes** sobre `triang`.

(b) **15** **tests** en `TriTypTest`.

(c) **Mutation score: 100 % (39/39)**. Progreso:

- Línea base inicial (1 test): 23 % (9/39).
- Suite final (15 tests): 100 % (39/39).

(d) **0** **mutantes equivalentes observados**: el reporte final de PIT terminó sin sobrevivientes (`KILLED 39 / GENERATED 39`), así que no quedó ningún mutante vivo para analizar.

2.4. Cierre

Métrica	Valor
Mutantes generados	39
Mutantes muertos	39
Mutantes equivalentes observados	0
Score bruto	100 %

3. Ejercicio 3: fuzzing y bc

Completar la parte de fuzzing del template: implementar `Mutator`, `MutationFuzzer` y `RandomFuzzer`, y cerrar el test parametrizado que ejecuta `bc` sobre entradas generadas por fuzzing.

3.1. Mutator

Tres operadores básicos sobre strings:

- `deleteRandomCharacter(s)` — borra un carácter en posición aleatoria.
- `insertRandomCharacter(s)` — inserta un carácter ASCII aleatorio (rango `[32, 126]`) en una posición aleatoria.
- `flipRandomCharacter(s)` — selecciona un carácter al azar y le da vuelta uno de los 7 bits bajos.

El método `mutate(s)` elige uno de los tres con probabilidad uniforme:

```
1 public static String mutate(String s) {
2     if (s == null) {
3         throw new IllegalArgumentException("Input string cannot
4             be null");
5     }
6     int choice = RANDOM.nextInt(3);
7     switch (choice) {
8         case 0: return deleteRandomCharacter(s);
9         case 1: return insertRandomCharacter(s);
10        default: return flipRandomCharacter(s);
11    }
12 }
```

Las tres operaciones validan `null` y manejan el caso de string vacío donde corresponde (no se puede borrar ni flippear un carácter de una cadena de longitud 0).

3.2. MutationFuzzer

Corregí un bug del constructor (el parámetro `max_mutations` quedaba mal asignado) y completé `fuzz()`:

1. elige aleatoriamente una semilla de la población inicial,
2. determina cuántas mutaciones aplicar entre `min_mutations` y `max_mutations`,

3. aplica las mutaciones en cadena (cada nueva mutación sobre el resultado de la anterior) y devuelve la cadena final.

```
1 public String fuzz() {
2     String candidate = population.get(random.nextInt(population.
3         size()));
4     int mutations = min_mutations;
5     if (max_mutations > min_mutations) {
6         mutations += random.nextInt((max_mutations -
7             min_mutations) + 1);
8     }
9     for (int i = 0; i < mutations; i++) {
10        candidate = Mutator.mutate(candidate);
11    }
12    return candidate;
13 }
```

Así se generan entradas progresivamente más alejadas de las semillas, lo que ayuda a explorar caminos que un fuzzer puramente aleatorio difícilmente alcanzaría.

3.3. RandomFuzzer

`fuzz()` genera cadenas completamente aleatorias: longitud entre 0 y `maxLength`, caracteres en el rango `[charStart, charStart + charRange)`. Sin estructura previa. Sirve como contrapunto al fuzzer por mutación.

```
1 public String fuzz() {
2     int stringLength = random.nextInt(maxLength + 1);
3     StringBuilder result = new StringBuilder(stringLength);
4     for (int i = 0; i < stringLength; i++) {
5         char randomChar = (char) (charStart + random.nextInt(
6             charRange));
7         result.append(randomChar);
8     }
9     return result.toString();
10 }
```

3.4. LinuxCommandTest

Test parametrizado que corre `bc` con cadenas generadas por el `MutationFuzzer`. La semilla inicial es `"2 + 2"` y se hacen 100 trials.

El criterio de validación está orientado a detectar fallas **graves**, no a esperar una salida específica:

- el proceso no debe terminar con códigos típicos de aborto o segfault (134, 139);
- `stderr` no debe contener `segmentation fault`, `core dumped` ni `illegal instruction`.

```
1 int exitCode = process.waitFor();
2
3 assertEquals(134, exitCode); // abort
4 assertEquals(139, exitCode); // segfault
5 String stderrLower = stderr.toString().toLowerCase();
6 assertFalse(stderrLower.contains("segmentation fault"));
7 assertFalse(stderrLower.contains("core dumped"));
8 assertFalse(stderrLower.contains("illegal instruction"));
```

Importante: no se exige `exitCode == 0`. Es esperable que `bc` rechace muchas entradas fuzzed devolviendo errores sintácticos — ese comportamiento es manejo normal de entradas inválidas, no una falla del programa.

3.5. Verificación

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true test
```

Resultado: BUILD SUCCESS, LinuxCommandTest 100/100, TriTypTest 15/15.